

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
DEPARTAMENTO DE INFORMÁTICA APLICADA
INF01120 – DESENVOLVIMENTO DE SOFTWARE
PROFESSOR: Marcelo Soares Pimenta (mpimenta@inf.ufrgs.br)

Enunciado do Trabalho Prático – Fase 1

Introdução

LEIAM ATÉ O FIM este enunciado !!!!!

Este trabalho prático será desenvolvido em grupo ao longo do semestre. O objetivo principal é **aplicar NA PRÁTICA os conceitos de engenharia de software** estudados na disciplina: boas práticas de programação, identificação e eliminação de *bad smells*, programação orientada a objetos, refatoração, modularidade e princípios SOLID. O domínio do problema foi escolhido por ser simples e não exigir conhecimento aprofundado, permitindo que o foco seja a **qualidade do código** produzido.

Atenção: mais importante do que o programa “funcionar perfeitamente” é que ele seja **organizado, modular, extensível e fácil de entender**. Um código bem estruturado, mesmo com funcionalidades parciais, será mais valorizado do que um código completo, mas caótico.

No desenvolvimento real de software, os requisitos mudam. Clientes, usuários ou o mercado impõem novas necessidades, e o sistema precisa evoluir. Para simular essa realidade, **os requisitos deste trabalho não serão totalmente estáveis**. Ao longo do semestre, o professor irá introduzir **mudanças** (tanto funcionais quanto não funcionais) em momentos previamente definidos. O grupo deverá adaptar o sistema para acomodar essas mudanças, aplicando os princípios de design que favoreçam a extensibilidade e a manutenibilidade.

Descrição:

O trabalho será dividido em **fases**, e em cada fase poderá haver um conjunto de novas funcionalidades ou restrições a serem incorporadas. O grupo deve projetar o software de forma que ele seja preparado para essas alterações – ou seja, deve ser fácil adicionar novos comportamentos sem modificar o código existente (princípio Aberto/Fechado).

Você deverá implementar um **gerador de sequências musicais a partir de um texto de entrada**. O sistema receberá um arquivo de texto (por exemplo, um conto, uma notícia, um poema) e, com base em regras de mapeamento, produzirá uma sequência de notas musicais (ou eventos) que poderá ser reproduzida através de uma biblioteca existente na linguagem escolhida. Podemos dizer informalmente que o software ‘toca’ (via acionamento de funções de som) um conjunto de NOTAS de acordo com o TEXTO segundo alguns PARÂMETROS (timbre, ritmo – na forma de Beats por Minuto ou BPM, etc). Os parâmetros são definidos através de um mapeamento de texto para informações musicais. **Este mapeamento é apenas o ponto de partida;** mudanças ocorrerão.

O mapeamento inicial proposto é o seguinte e deve ser seguido À RISCA:

Texto	Informação Musical ou Ação
(Letra maiúscula) A	Nota Lá
(Letra maiúscula) B	Nota Si
(Letra maiúscula) C	Nota Dó
(Letra maiúscula) D	Nota Ré
(Letra maiúscula) E	Nota Mi
(Letra maiúscula) F	Nota Fá
(Letra maiúscula) G	Nota Sol
Letra (maiúscula) H	Nota Si Bemol
Letras a,b,c,d,e,f,g,h minúsculas	NÃO são notas e sim Silêncio ou Pausa
Caractere Espaço	Aumenta volume para o DOBRO do volume; Se não puder dobrar, coloca o valor máximo;

Texto	Informação Musical ou Ação
Caractere ! (ponto de exclamação)	Trocar instrumento para o instrumento General MIDI #24 (Bandoneon)
Qualquer outra letra vogal (O ou o, I ou i, U ou u)	Trocar instrumento para o instrumento General MIDI #110 (Gaita de Foles)
Qualquer outra letra consoante (todas consoantes exceto as que são notas)	Se caractere anterior era NOTA (A a H), repete nota; Caso contrário, Silêncio ou pausa
Dígito par	Trocar instrumento para o instrumento General MIDI cujo número é igual ao valor do instrumento ATUAL + valor do dígito
Caractere ? (ponto de interrogação) e caractere . (ponto)	Aumenta UMA oitava; Se não puder, aumentar, volta à oitava default (de início)
Caractere NL (nova linha)	Trocar instrumento para o instrumento General MIDI #123 (Ondas do Mar)
Caractere ; (ponto e vírgula) OU Dígito ímpar	Trocar instrumento para o instrumento General MIDI #15 (Tubular Bells)
Caractere , (vírgula)	Trocar instrumento para o instrumento General MIDI #114 (Agogô)
ELSE (nenhum dos caracteres anteriores)	Se caractere anterior era NOTA (A a H), repete nota; Caso contrário, Silêncio ou pausa

O texto que serve como entrada DEVE ser entrado via um campo texto na interface do software .
Para a saída sonora , todos podem usar alguma API ou biblioteca de funções de som, como JavaSound ou JFugue (linguagem Java), ou PyGame (Python). O grupo pode escolher a API.

Requisitos Funcionais Iniciais (Mínimos): O sistema deve....

1. Ler o texto de entrada num campo texto na interface do software.
2. Interpretar caractere por caractere, aplicando as regras de mapeamento definidas.
3. Gerar uma saída musical reproduzível, ou seja, deve ser possível ouvir o resultado.
4. O usuário deve poder configurar alguns parâmetros iniciais (como BPM – batidas por minuto, instrumento inicial, oitava padrão, volume, etc).

Requisitos Não-Funcionais Iniciais (Ênfase na Qualidade do Código)

- **Modularidade:** o sistema deve ser dividido em módulos com responsabilidades bem definidas. Por exemplo: módulo de leitura do texto, módulo de interpretação (aplicação das regras), módulo de geração de eventos musicais, módulo de saída (MIDI, áudio, etc.).
- **Baixo acoplamento e alta coesão:** as classes devem ter poucas dependências entre si e cada uma deve representar um conceito coeso.
- **Aplicação dos princípios SOLID:**
 - **S** – Responsabilidade Única: cada classe deve ter apenas uma razão para mudar.
 - **O** – Aberto/Fechado: o sistema deve ser extensível para novas regras ou novos formatos de saída sem modificar o código existente.

- **L** – Substituição de Liskov: as subclasses devem poder substituir suas classes base sem alterar o comportamento esperado.
- **I** – Segregação de Interfaces: interfaces específicas são melhores que uma interface genérica.
- **D** – Inversão de Dependência: depender de abstrações, não de implementações concretas.
- **Legibilidade e boas práticas:** nomes significativos, comentários pertinentes, formatação consistente, ausência de código duplicado.
- **Testabilidade:** o código deve ser escrito de forma a permitir testes automatizados (unitários) sempre que possível.
- **Versionamento:** o grupo deve utilizar o repositório GitHub criado para o grupo para controle de versão e colaboração. O histórico de *commits* deve refletir o desenvolvimento progressivo.

Mudanças de Requisitos ao Longo do Semestre

Para simular a evolução de um projeto real, o professor introduzirá **mudanças de requisitos** em momentos específicos. As mudanças serão comunicadas em aula e publicadas no Moodle. Os grupos deverão adaptar seu sistema para incorporá-las, mantendo a qualidade do código.

O grupo deve estar preparado para essas mudanças. A avaliação considerará não apenas se as novas funcionalidades foram implementadas, mas principalmente **o quão fácil foi incorporá-las** e se o design original permitiu essa extensão sem grandes refatorações.

A bibliografia básica inclui exemplos de parâmetros que podem controlar a performance:

- Ver descrição em : <http://tones.wolfram.com/about/controls-side.html>
- Ver explicação na wikipedia sobre notação musical:
“Notation is the written expression of music notes and rhythms on paper using symbols. When music is written down, the pitches and rhythm of the music is notated, along with instructions on how to perform the music.” (<http://en.wikipedia.org/wiki/Music#Notation>)
- Ver descrição mais completa de parâmetros em http://en.wikipedia.org/wiki/Music_theory – ver Fundamentals of Music
- O código dos instrumentos (piano, “telefone tocando”, etc) está definido no General MIDI (ver https://pt.wikipedia.org/wiki/General_MIDI)

O trabalho pode ser dividido em partes:

1a Parte: O grupo deve estender a **lista inicial dos requisitos** que o sistema deve possuir no ponto de vista dos autores do projeto. Cada requisito é um item da lista acima (informal, em português) e pode ser do tipo funcional (relativo a funcionalidade do sistema) ou não-funcional (relativo a características de performance, usabilidade, cronograma, orçamento, limitações tecnológicas, entre outras). Esta lista serve para estabelecer quais operações o sistema permitirá que sejam realizadas sobre quais elementos, deixando claro quais as consequências (alteração de aparência, mensagens ao usuário, etc).

2a Parte: Nesta parte, o grupo deve **PROJETAR A PARTE FUNCIONAL** do sistema, preferencialmente **DEFININDO CLASSES** – com atributos e métodos - correspondentes a Tipos Abstratos de Dados que fazem parte do programa. Não se exige uma notação específica. A definição (em qualquer sintaxe) das classes deve conter os atributos e métodos (mesmo que depois isto mude, coloquem ao menos os definidos até agora). Classes de Interface com usuário **NÃO** precisam ser definidas em detalhes agora.

3a Parte: Nesta parte, o grupo deve **PROJETAR** um rascunho inicial da **INTERFACE COM O USUÁRIO** do sistema. Para isto, deverá refletir sobre quais tarefas principais serão feitas pelo usuário ao usar o sistema e como serão feitas. Além disto, a documentação entregue deverá incluir uma impressão (ou **croqui/maquete**) de cada tela/unidade de apresentação por página, seguida das explicações/argumentações que justifiquem seu *layout* e seu comportamento.

Nas fases seguintes do trabalho, o grupo irá **IMPLEMENTAR** um protótipo do sistema projetado. **MAS SUGIRO** que o grupo comece O QUANTO ANTES a implementação, para ter uma ideia dos desafios a serem vencidos. Não é definida *a priori* nenhuma tecnologia para ser usada mas o grupo pode alternativamente escolher a linguagem e a plataforma mais adequados para seu desenvolvimento, obviamente

compatível com os elementos e restrições definidos pelo professor. Reuso de algum código existente é tolerado, mas deve ser sinalizado na documentação e as razões de sua utilização deverão ser explicadas ao professor na documentação entregue. Ao implementar o protótipo com o máximo possível de funções que foram projetadas, não esquecer de TESTAR e DEPURAR o que foi implementado ANTES de demonstrá-lo ao professor. Todo o conteúdo necessário para o desenvolvimento será visto na disciplina.

Mais perto do final da disciplina, o grupo irá APRESENTAR as características principais de seu projeto e DEMONSTRAR o sistema para o professor. Cada grupo poderá ser convidado a apresentar as idéias principais de seu trabalho para a turma (20 min no máximo por apresentação). Além disto, a versão implementada do sistema deverá ser apresentada ao professor em uma sessão especial de demonstração em data a ser definida. Assume-se que a documentação final do trabalho também será entregue ao professor na data definida. O professor não apenas fará uso da documentação para utilizar o sistema e avaliá-lo mas também poderá questionar oralmente os componentes do grupo a respeito de todas as partes do trabalho. Deve ficar clara nas respostas a participação e o engajamento de todos os indivíduos no trabalho do grupo.

Observações:

O trabalho FINAL (ao fim das fases 1,2 e 3) a ser entregue inclui a documentação associada a TODAS as partes e deve conter também a identificação do grupo (número do grupo, nomes de todos componentes) e todas as suposições feitas durante a realização do trabalho. Definições mais precisas que porventura sejam necessárias serão acrescentadas no decorrer do semestre. Em caso de dúvidas, consulte primeiro a bibliografia disponível (preferencialmente), depois pesquise em outras bibliografias (use a biblioteca e a Internet) e, em caso de necessidade, consulte o professor pessoalmente ou via e-mail. **A disciplina prevê algumas aulas de reuniões com o professor para acompanhamento dos projetos e esclarecimento das dúvidas surgidas. As datas serão divulgadas em aula e via moodle.** Em caso de reunião fora do período de aula, lembre-se de agendar com o professor (via email) para garantir que será atendido. Sem agendamento prévio, a prioridade do professor será sempre atender aos compromissos agendados previamente. Bom trabalho!!

Datas importantes SERÃO DEFINIDAS EM AULA e DIVULGADAS VIA MOODLE

A Entrega da FASE 1 consiste de:

- Definições de classes (sem sintaxe específica, apenas listem as classes, atributos e métodos que estiverem definidos)
- Croquis (maquetes) de interface com usuário, de todas telas previstas
- Lista de requisitos funcionais: uma lista informal do que o software irá fazer (para ficar claro o que vcs têm em mente).

NO ENTANTO, mesmo que NÃO seja necessária a entrega de alguma implementação nesta FASE1, o professor **aconselha que tentem implementar as classes e começar a construir o código**, pois MUITAS dúvidas surgem e MUITAS MUDANÇAS decorrem disto.

Além disto, AULAS DE ACOMPANHAMENTO serão agendadas para resolução de dúvidas ou discussão do trabalho com os colegas e o professor. Uma vez decididas, as datas serão postadas no moodle da disciplina.